

# Q-ROVER

## 1차시

2026-2 HYPER 자율주행트랙

9/14 (월) 18:30-20:00

전정대 539호



# 들어가기: 로봇 개발의 과정?

## 하나의 종합 예술

로봇 하나를 만드는 데에는 **EE, SW, HW, AI**가 전부 들어갑니다

## 서로가 서로의 제약 조건

모터 무게(HW)는 배터리 용량(EE)에 영향을 주고, 센서 통신 딜레이(EE)는 제어 루프 주기(SW)를 제한합니다.

## 경계를 허무는 시스템 엔지니어링

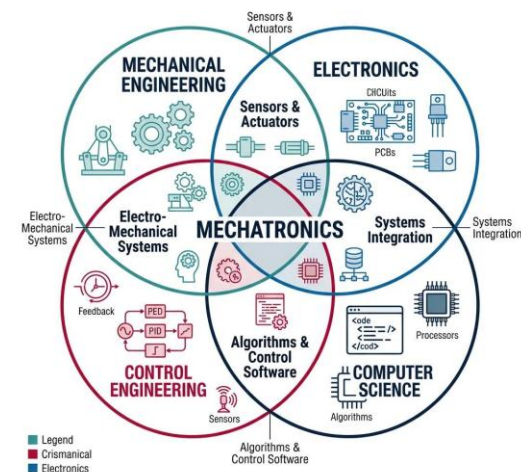
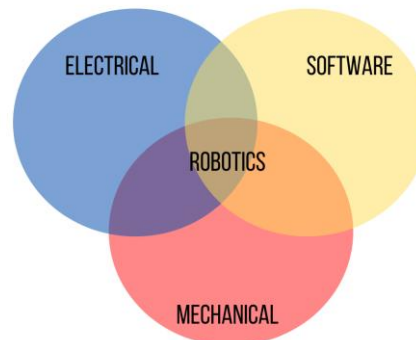
단일 전공 지식만으로는 한계가 발생하며, 하드웨어의 물리적 특성을 이해하고 코드를 작성하는 **전체적인 시각**이 요구됩니다.

## 앞으로 다룰 내용

큐로버를 움직이기 위해 이 네 가지 영역이 어떻게 맞물려 동작하는지 살펴봅니다.



ex<HYPERNOVA>



# 들어가기: 로봇을 만드는 도구들

## 프로그램 연결

ROS 2(미들웨어)

카메라·인식·주행을 메시지로 연결

## 시간 관리

Zephyr / FreeRTOS

제어 작업의 주기와 우선순위 관리



## 가상 환경에서 시험하고 학습하기

Isaac Sim은 로봇과 센서를 시뮬레이션

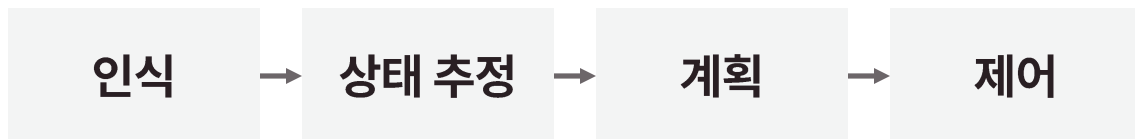
Isaac Lab은 그 위에서 학습

MuJoCo도 물리·접촉 실험에 사용됨

향후 트랙에서 만나요

# 들어가기: 데이터로 동작을 배우는 로봇

## 전통적 로봇틱스 파이프라인 (Sense-Plan-Act)

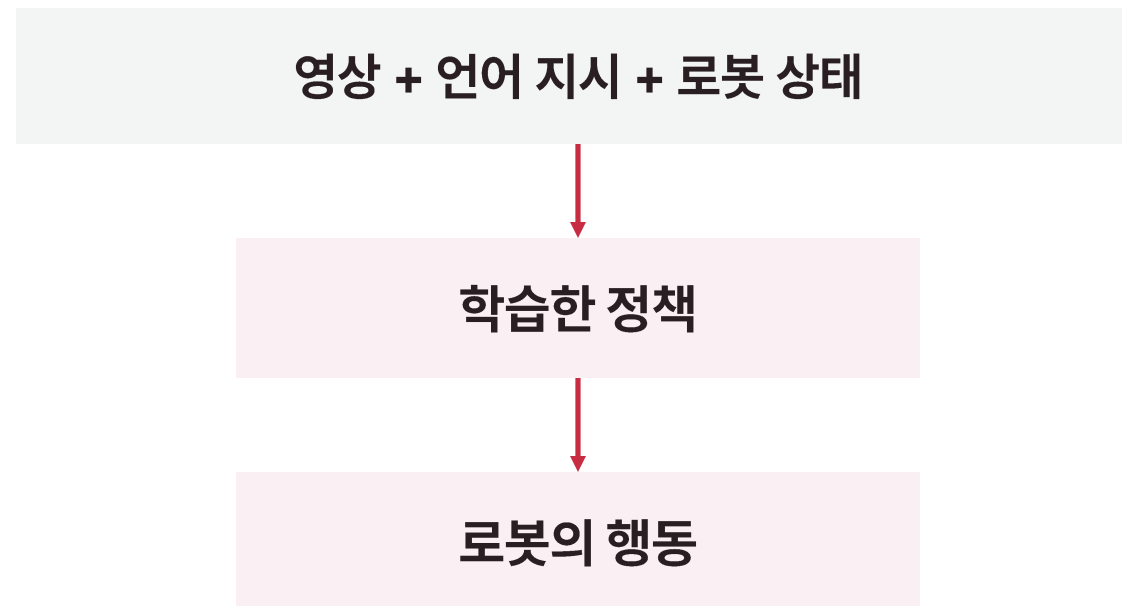


## 모방 학습 기반 제어 (Imitation Learning)

작업 시범: 원격조작 및 모션 캡처 기반 인터페이스

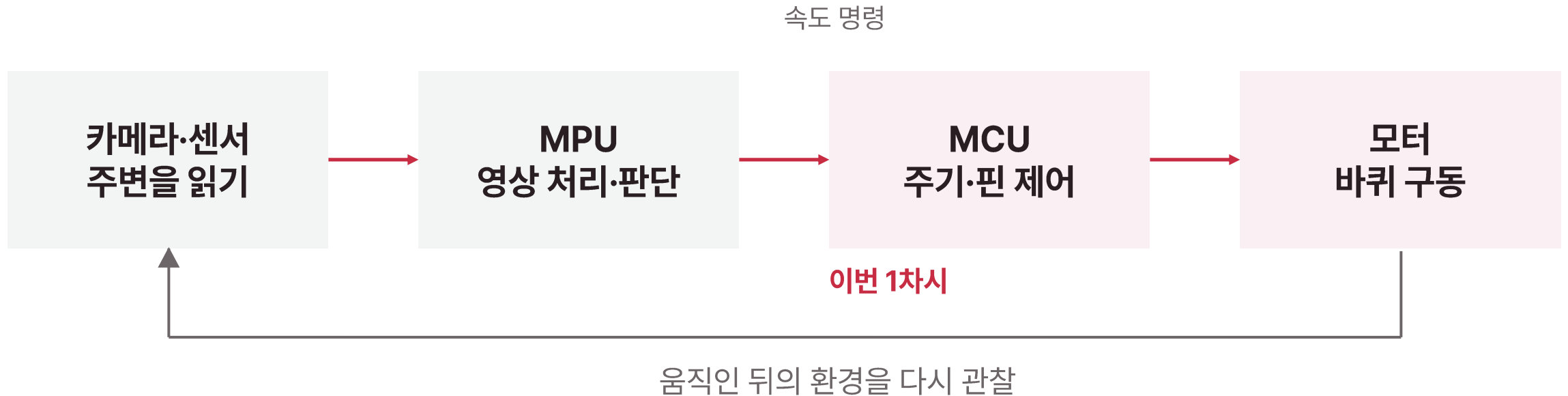
정책 학습: ACT(트랜스포머 기반) 및 확산 모델 중심

## 엔드투엔드 파운데이션 모델 (VLA)



RT-2, Gemini Robotics, Unitree UnifoLM  
이런 흐름으로 이어지고 있습니다.

# 들어가기: 큐로버 안의 전체 구조

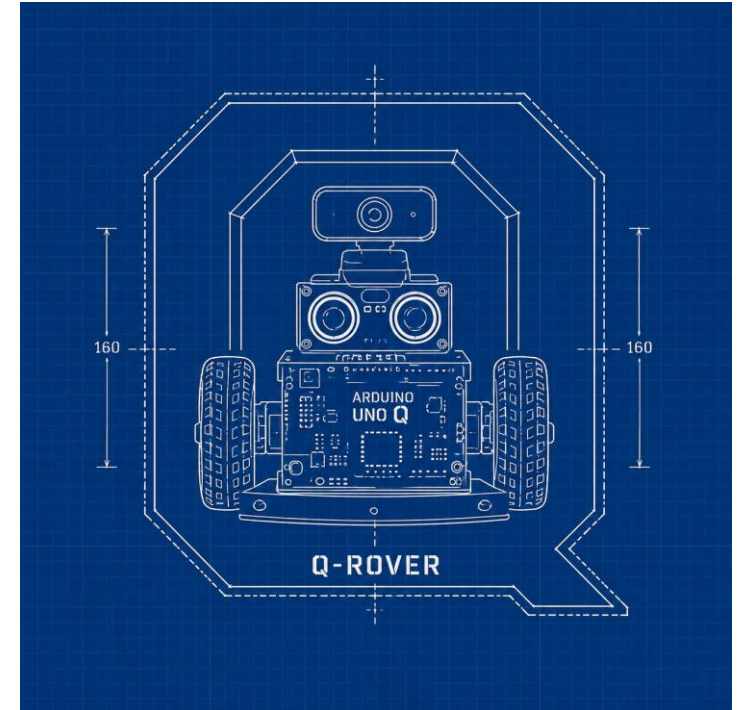


오늘은 MCU와 모터를 다룹니다. 카메라와 판단은 다음 차시에서 이어집니다.

# 들어가기: 큐로버의 방식

**체급은 소형화되었지만, 제어의 본질은 다르지 않습니다**

아두이노 기반의 소형 RC카로 시작하지만, 감각-판단-제어로 이어지는 시스템의 큰 틀은 나중에도 꼭 써먹게 됩니다.

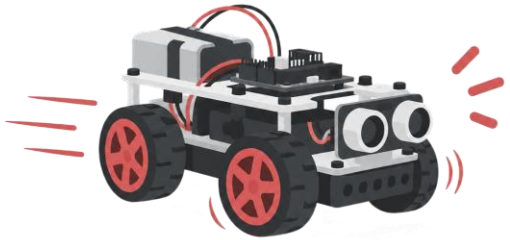


# 작동하는 로버로 바로 시작합니다

## 진행 과정

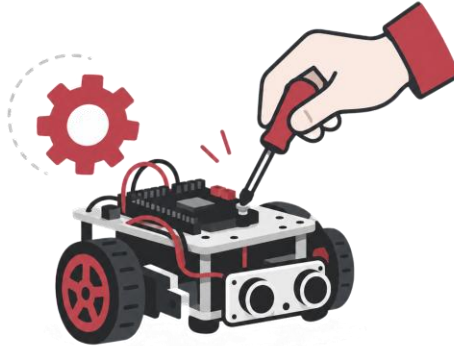
01

작동하는 것을 쓴다



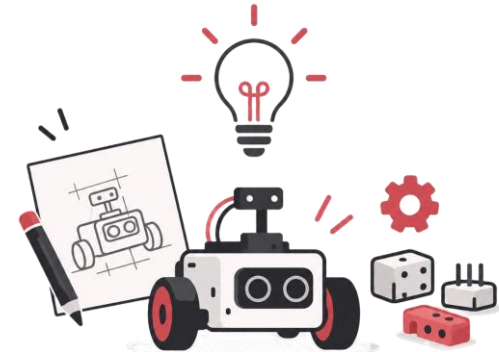
02

원하는 대로 고쳐 본다

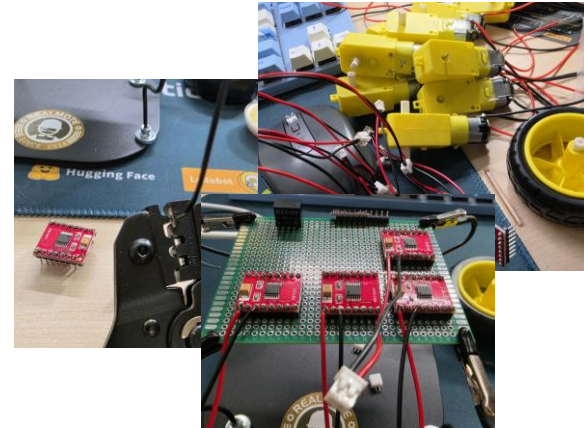


03

내 아이디어를 만든다



납땜·조립·배선은 미리 준비했습니다. 처음부터 움직이는 로버로 시작합니다.



# 오늘의 주행 목표

전진 1초  
급정지  
제자리 한 바퀴  
정지



주행 영상

# Arduino UNO Q 소개

보드 한 장 안에  
컴퓨터와 마이크로컨트롤러

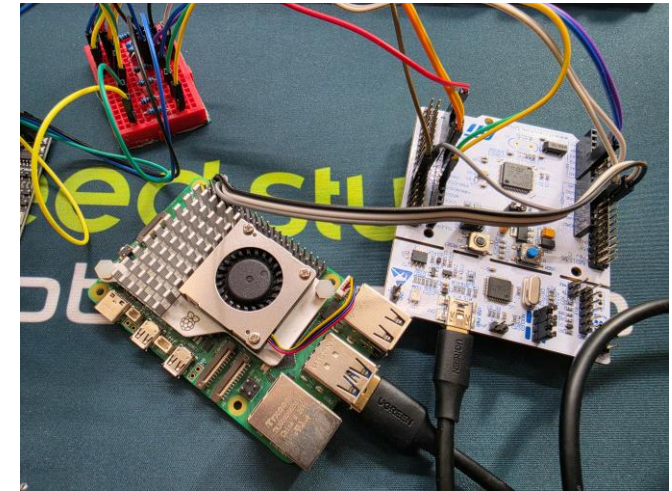
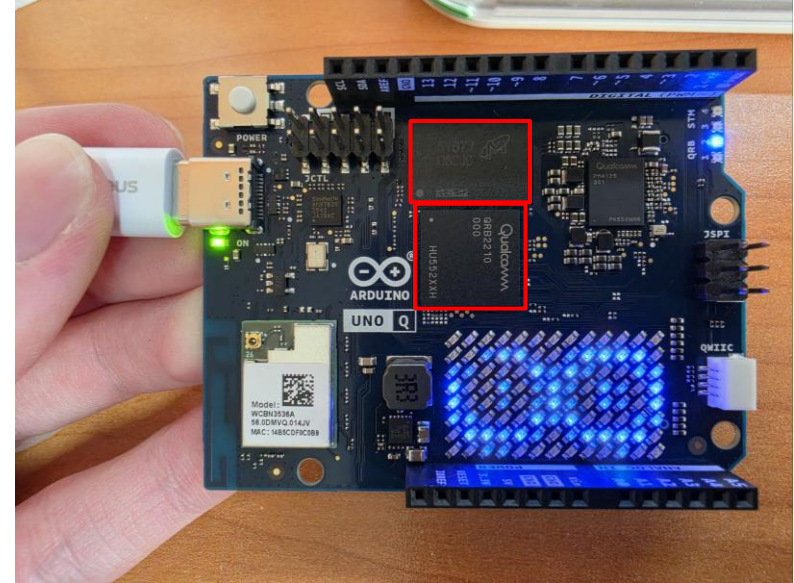
## Qualcomm QRB2210(MPU)

Debian Linux, 영상 처리와 판단  
rasberry pi, jetson, LattePanda 등이 있음

## STM32U585(MCU)

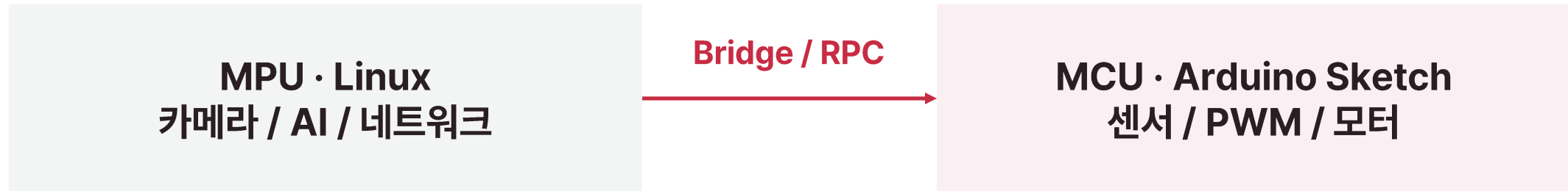
Zephyr 위의 Arduino Sketch를 구동  
오늘 사용할 모터 제어 코드가 돌아가는 위치  
아두이노, esp32, Raspberry Pi Pico, stm32, teensy 등

오늘은 STM32에서 실행하는 Sketch를 수정합니다.



이렇게 두 보드가 해야 할 일을 보드 하나로 사용

# 한 보드 안에서 역할 나누기



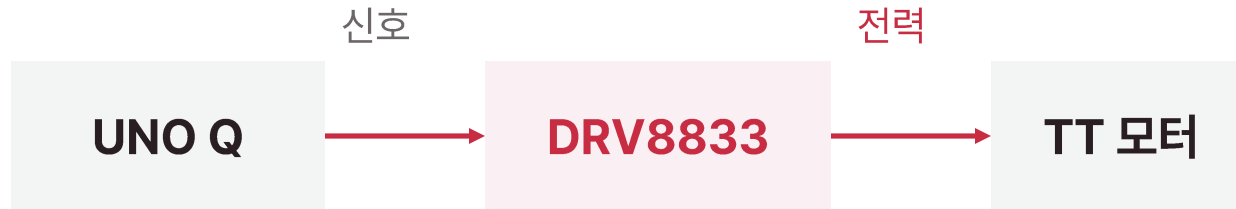
| 구분           | MPU(Microprocessor Unit) | MCU(Microcontroller Unit) |
|--------------|--------------------------|---------------------------|
| 메모리 구성       | 외부 RAM·Storage와 사용       | RAM·Flash를 칩에 포함          |
| 주요 역할        | 영상 처리와 복잡한 연산            | 주기적인 제어와 핀 출력             |
| UNO Q의 실행 환경 | Debian Linux             | Zephyr OS                 |

간단히 말해 MPU에서 상위 로직을 제어하고 MCU에서 하위 로직을 제어한다고 보면 됩니다.

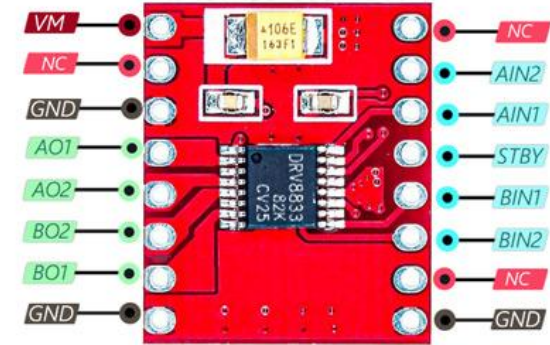
# 모터 드라이버가 하는 일

MCU에서 GPIO\*로 움직임 신호를 전달하면  
드라이버가 전력을 전달

보드에 있는 핀(3.3V, 5V)만으로는 모터를 돌리는 데  
필요한 전류를 공급할 수 없습니다.



VM  
GND  
CONTROL PINS  
NO CONNECTED  
MOTOR PINS



www.Electropeak.com



DRV8833으로 두 모터의 정회전·역회전과 PWM 구동을 제어합니다.

\*GPIO : 외부 장치와 디지털 신호를 주고받는 범용 입출력 핀

# Q-Rover의 배선

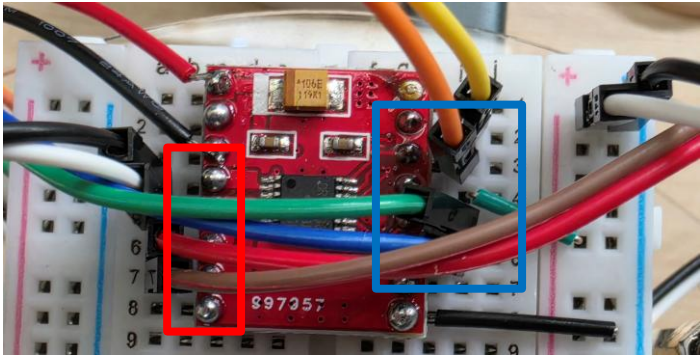
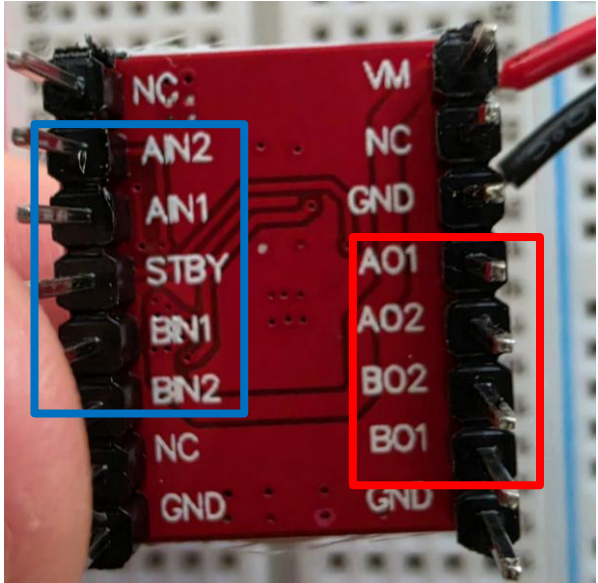
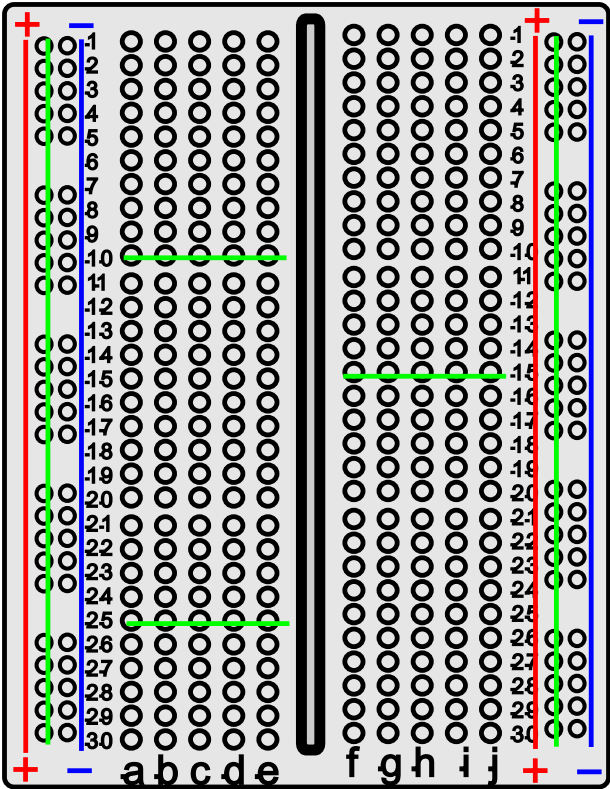
| DRV8833   | 연결              | 역할        |
|-----------|-----------------|-----------|
| AIN1      | UNO Q D5        | 왼쪽 모터 제어  |
| AIN2      | UNO Q D3        | 왼쪽 모터 제어  |
| BIN1      | UNO Q D6        | 오른쪽 모터 제어 |
| BIN2      | UNO Q D9        | 오른쪽 모터 제어 |
| A01 / A02 | 왼쪽 모터의 두 선      | 모터에 직접 연결 |
| B01 / B02 | 오른쪽 모터의 두 선     | 모터에 직접 연결 |
| GND       | UNO Q GND       | 공통 기준     |
| VM / GND  | AA 배터리 4구 + / - | 모터 전용 전원  |



이번 실습 배선은 D3·D5·D6·D9입니다. 코드의 핀 번호와 실제 배선을 맞춥니다.

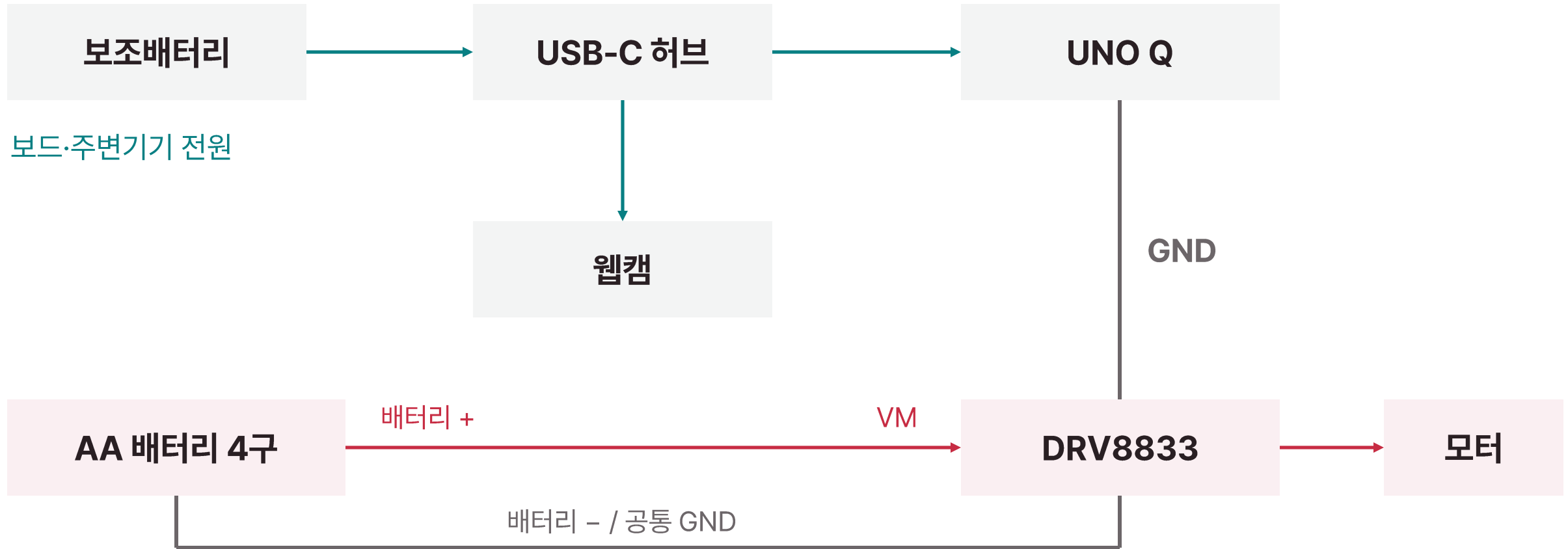
# 지금 한번 연결해 봅시다

| DRV8833   | 연결              |
|-----------|-----------------|
| AIN1      | UNO Q D5        |
| AIN2      | UNO Q D3        |
| BIN1      | UNO Q D6        |
| BIN2      | UNO Q D9        |
| A01 / A02 | 왼쪽 모터의 두 선      |
| B01 / B02 | 오른쪽 모터의 두 선     |
| GND       | UNO Q GND       |
| VM / GND  | AA 배터리 4구 + / - |



D3·D5·D6·D9, 모터 선 4개를 빼웠습니다. 팀원과 함께 브레드보드에 연결 해 봅시다

# 전원 분리와 공통 GND(-)



모터 기동 전류의 영향을 줄이려고 전원을 나누고, 신호의 기준을 맞추려고 GND를 연결해줬습니다.

# 실행 전, 잠깐 예측해 보기

```
// 바퀴 하나 굴리기 (speed: -255~255)
// speed > 0: 앞으로, speed < 0: 뒤로
// speed = 0: 힘 빼기 (Coast)
void wheel(int in1, int in2, int speed) {
  if (speed > 0) {
    analogWrite(in1, 255);
    analogWrite(in2, 255 - speed);
  } else if (speed < 0) {
    analogWrite(in2, 255);
    analogWrite(in1, 255 + speed);
  } else {
    analogWrite(in1, 0);
    analogWrite(in2, 0);
  }
}
```

## 팀에서 1분만 이야기해 봅시다

- 01 wheel(in1, in2, 255)와 wheel(in1, in2, 60)  
두 핀에 쓰는 analogWrite 값은 각각 얼마일까?
- 02 speed = 0과 speed = 1  
설정값은 (0, 0)에서 (255, 254)로 바뀐다.  
왜 0만 따로 처리했을까?
- 03 255 - speed  
speed를 키우면 IN2에 쓰는 값은 작아진다.  
그런데 왜 더 빨리 돌까?

03은 아직 답을 몰라도 괜찮습니다. 굴려 보고 PWM을 살펴본 뒤 다시 이야기해 봅시다.

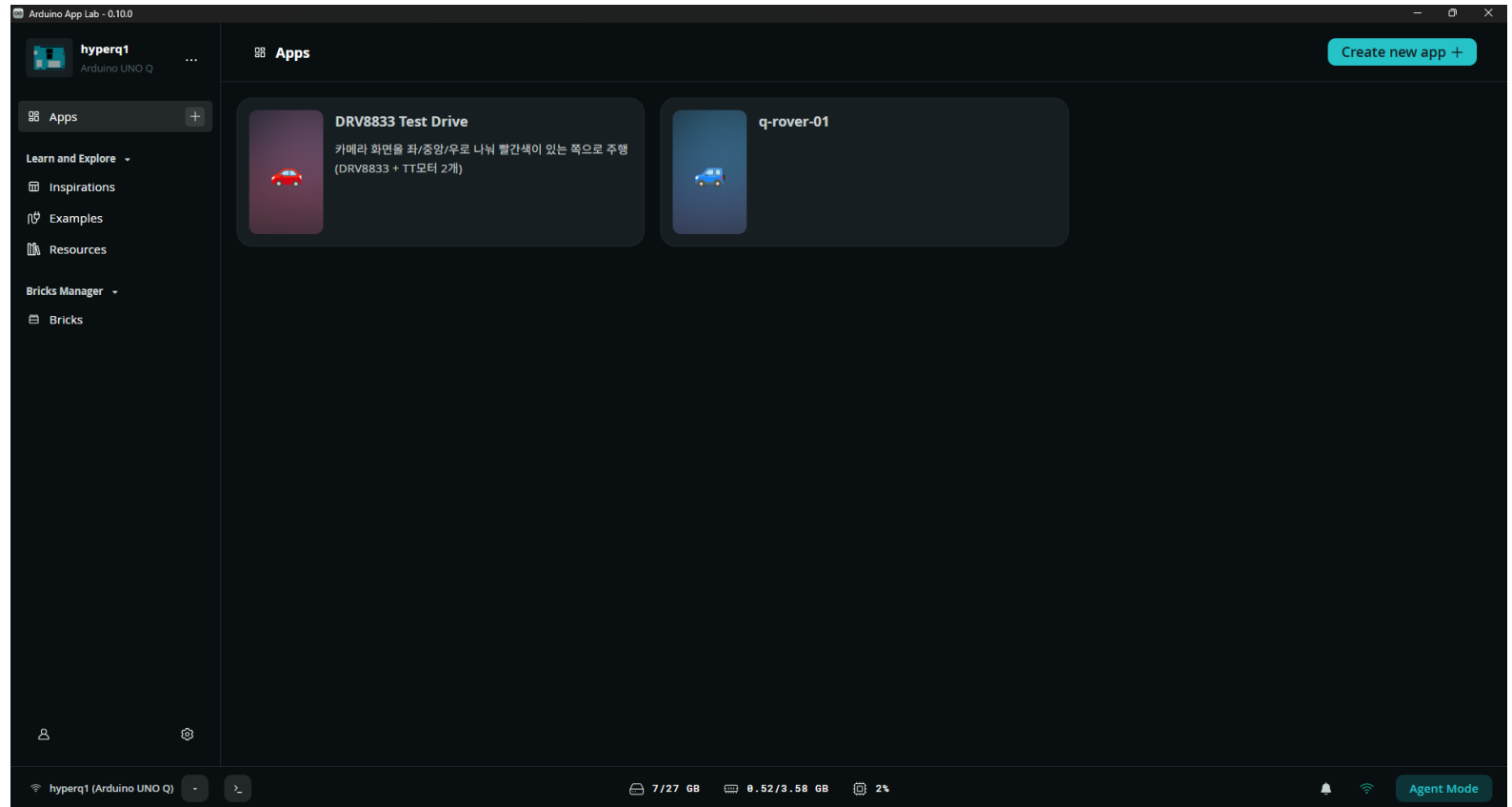
# App Lab에서 첫 실행

1 App Lab 실행  
(유선 연결 후 팀별로 핫스팟 연결)  
(보드 PW: hyper123)

2 Create new app  
(sketch.ino에 코드 추가)

3 Run 실행

Sketch 빌드  
STM32 업로드  
앱 실행



첫 실행은 바퀴를 바닥에서 띄운 상태로 확인합니다. 직진하지 않으면 하드웨어 핀이나 코드를 수정해서 해결해봅시다.

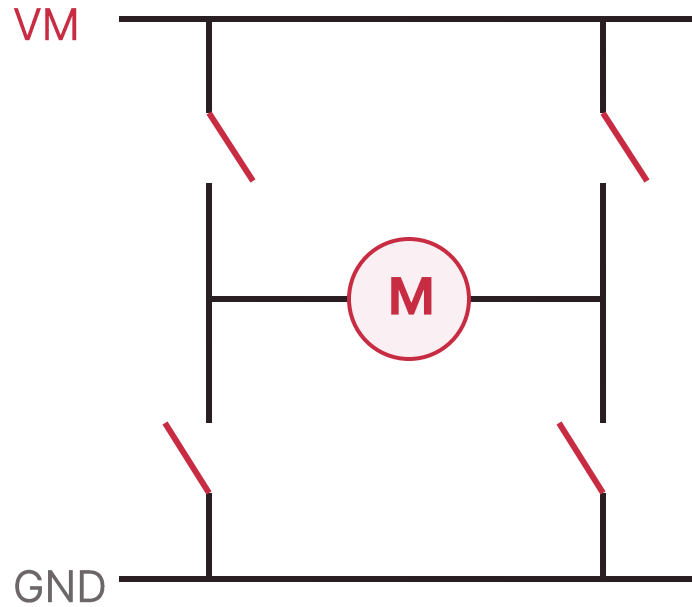
# 프로젝트 안에서 실행되는 코드



한 프로젝트에서 Linux 프로그램과 MCU 프로그램을 함께 관리합니다.

# 핀 두 개로 바퀴 하나 제어하기(모터 드라이버 방식)

## H-브리지 개념



4개의 스위치로  
양단의 전압을 바꿉니다

| IN1 | IN2 | 동작              |
|-----|-----|-----------------|
| 0   | 0   | Coast · 관성으로 정지 |
| 1   | 0   | Forward · 정방향   |
| 0   | 1   | Reverse · 역방향   |
| 1   | 1   | Brake · 전기적 제동  |

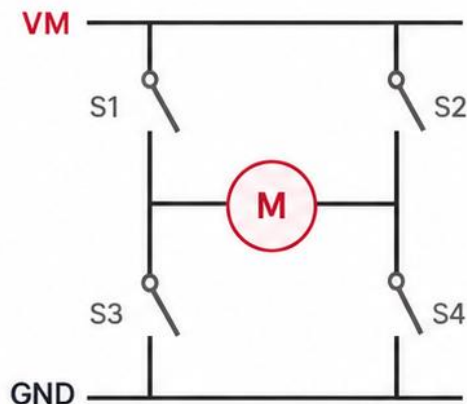
### 02번 답 · 0을 '힘 빼기'로 정했기 때문

(LOW, LOW)은 Coast, (HIGH, HIGH)는 Brake입니다.  
0도 양수 쪽 식에 넣으면 (255, 255), 즉 Brake가 됩니다.  
그래서 speed = 0일 때는 (0, 0)을 내보내도록 분리했습니다.

# H-브리지 4가지 동작

1. IN1 = 0, IN2 = 0

Coast / 관성 정지

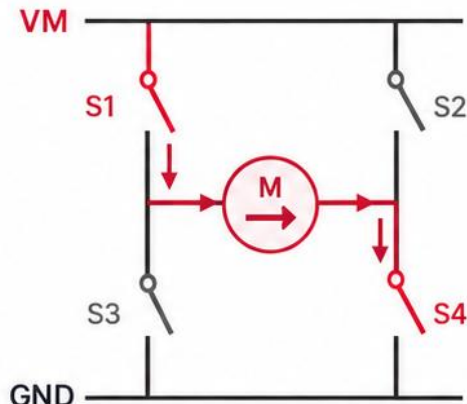


모터 단자 개방

모든 스위치가 OFF  
(모터가 전기적으로  
분리된 상태)

2. IN1 = 1, IN2 = 0

Forward / 정방향

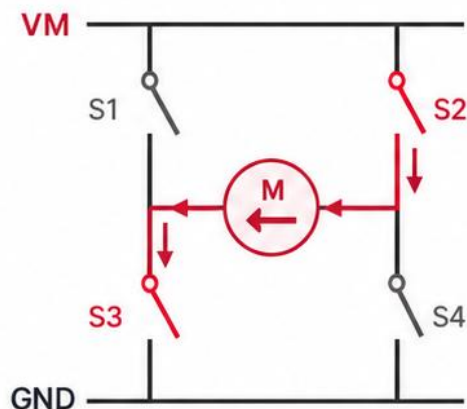


전류가 좌 → 우로 흐름

모터가 정방향으로  
회전합니다.

3. IN1 = 0, IN2 = 1

Reverse / 역방향

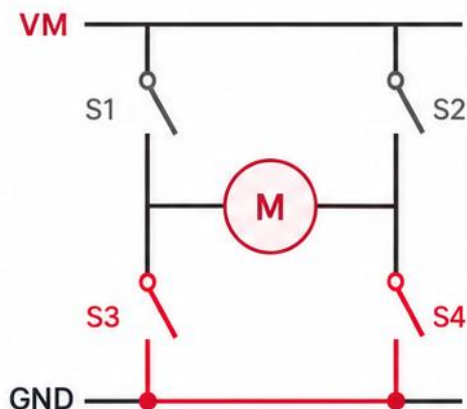


전류가 우 → 좌로 흐름

모터가 역방향으로  
회전합니다.

4. IN1 = 1, IN2 = 1

Brake / 전기적 제동



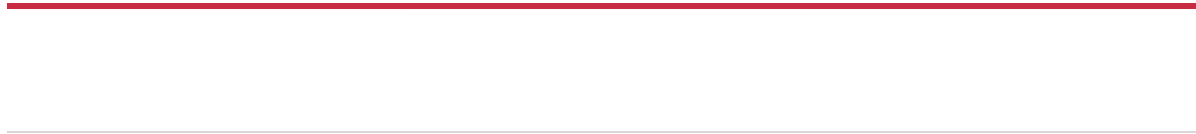
모터 양단을 같은 전위로  
묶어 제동

모터의 두 단자가 GND로  
연결되어 전기적 제동이  
걸립니다.

— 빨강 = ON / — 회색 = OFF

# PWM과 켜져 있는 시간

100%



`analogWrite(pin, 255)`

약 50%



`analogWrite(pin, 128)`

20%



`analogWrite(pin, 51)`

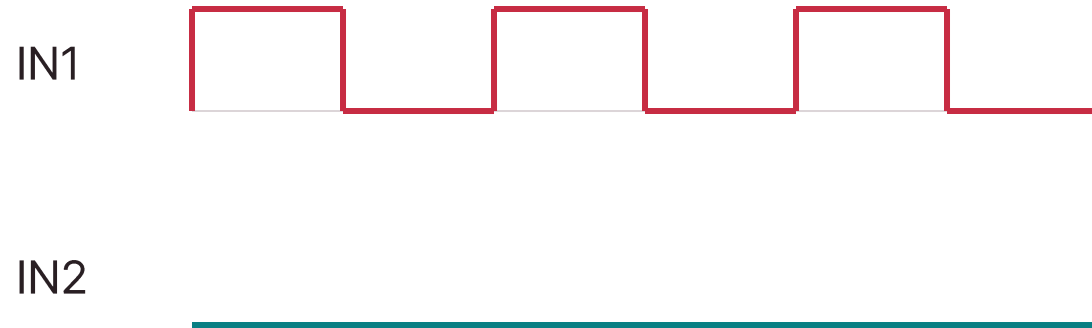
PWM은 HIGH로 켜져 있는 시간의 비율(duty)을 바꿉니다.

01번 답 · speed 255: (255, 0), speed 60: (255, 195). 두 값은 전압이 아닌 PWM 설정값입니다.

# PWM의 OFF 구간

## Fast decay(빠른 감쇄)

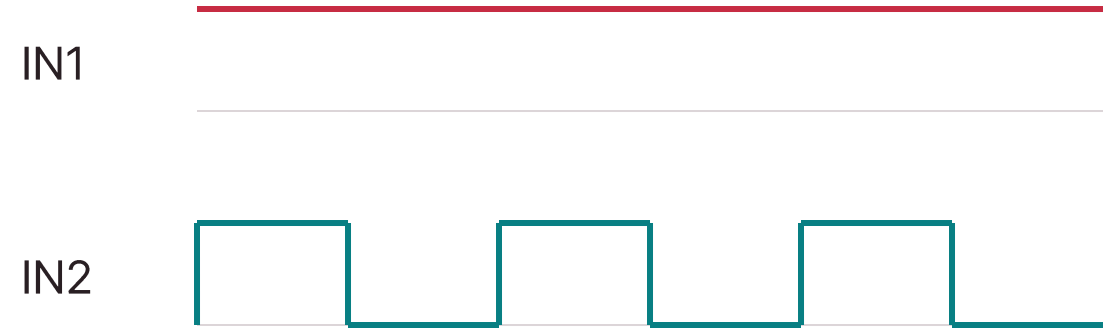
구동과 Coast를 반복



(1,0) 구동 / (0,0) Coast

## Slow decay(느린 감쇄) · 오늘의 방식

구동과 Brake를 반복



(1,0) 구동 / (1,1) Brake

감쇄는 전류가 줄어드는 방식입니다. 느린 감쇄는 저속 구동에 도움이 될 수 있습니다.

## 255 – speed의 의미

```
analogWrite(in1, 255);  
analogWrite(in2, 255 - speed);
```

**speed = 200**

IN2에 쓰는 값 = 55

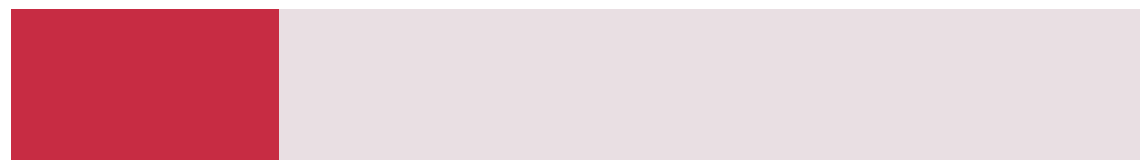


**구동 78%**

**Brake 22%**

**speed = 60**

IN2에 쓰는 값 = 195



**구동 24%**

**Brake 76%**

**03번 답 · 오타가 아니라 의도된 반전입니다.**

IN1이 HIGH일 때, IN2 설정값 / 255는 Brake 시간의 비율입니다.  
이 값이 작아지면 구동 시간이 늘어나, 더 빠르게 돌도록 구동합니다.

# wheel()과 brake()

```
void wheel(int in1, int in2, int speed) {  
  if (speed > 0) {  
    analogWrite(in1, 255);  
    analogWrite(in2, 255 - speed);  
  } else if (speed < 0) {  
    analogWrite(in2, 255);  
    analogWrite(in1, 255 + speed);  
  } else {  
    analogWrite(in1, 0);  
    analogWrite(in2, 0);  
  }  
}
```

```
void brake() {  
  analogWrite(AIN1, 255);  
  analogWrite(AIN2, 255);  
  analogWrite(BIN1, 255);  
  analogWrite(BIN2, 255);  
}
```

**양수는 정방향, 음수는 역방향**

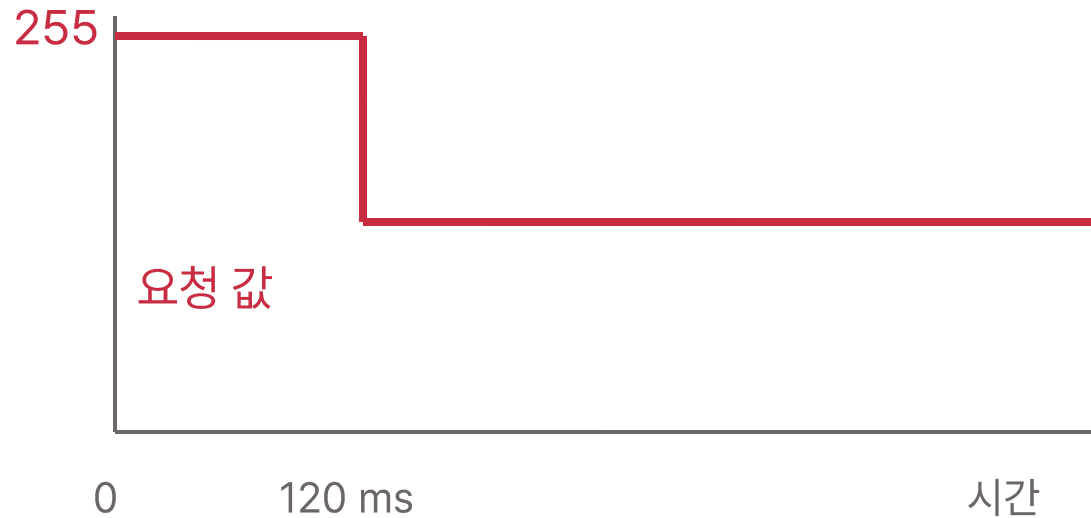
wheel(..., 0)은 Coast

brake()는 네 입력을 모두 HIGH

이 함수의 입력 범위는 -255~255입니다. 정지 방식은 호출하는 함수로 구분합니다.

# 출발할 때만 잠깐 강하게

## 기동 출력의 시간 흐름



정지 마찰과 기동 부하를 넘도록  
출발할 때만 최대 출력을 줍니다.

```
int kickOf(int v) {  
    return v > 0 ? 255 :  
           (v < 0 ? -255 : 0);  
}  
  
void startDrive(int l, int r) {  
    drive(kickOf(l), kickOf(r));  
    delay(120);  
    drive(l, r);  
}
```

120 ms는 이번 예제의 설정값입니다.  
배터리·바닥·하중에 따라 조정합니다.

# 미션: 전진, 급정지, 제자리 회전

```
void loop() {  
  startDrive(SPEED, SPEED);  
  delay(880);  
  brake();  
  delay(1000);  
  
  // TODO: 제자리에서 한 바퀴  
  startDrive( ???, ??? );  
  delay( ??? );  
  
  brake();  
  delay(3000);  
}
```

## 완료 조건

전진 1초

급정지

제자리 한 바퀴 ( $\pm 30^\circ$ )

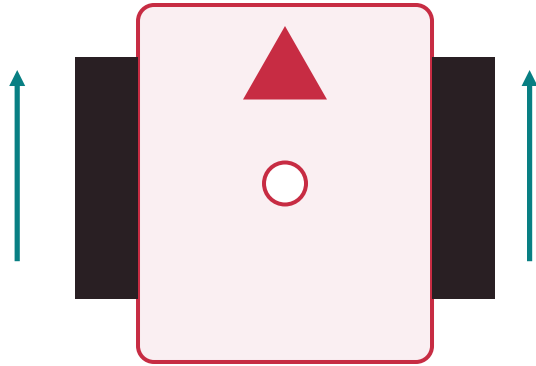
정지 후 반복

전진 구간

기동 120 ms + 대기 880 ms = 1초

# 제자리 회전과 시간 조정

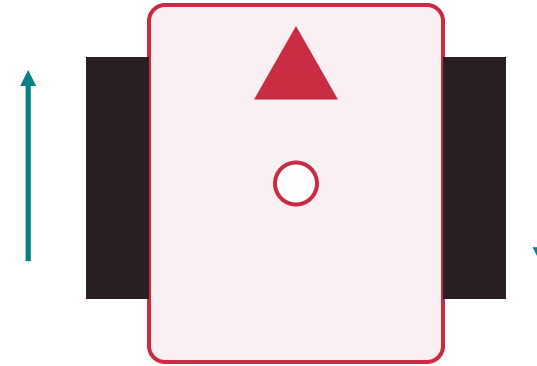
## 전진



왼쪽과 오른쪽을 같은 방향으로

**startDrive(140, 140)**

## 제자리 회전



왼쪽과 오른쪽을 반대 방향으로

**startDrive(140, -140)**

바닥에 시작 방향을 표시하고 회전 시간을 조금씩 조정합니다. 조건이 바뀌면 결과도 달라집니다.

# 오늘의 제출물

## 팀 리포지토리

---

**sketch/sketch.ino**

TODO를 채운 코드

---

**README.md**

팀원 이름

---

**demo.mp4**

10초 주행 영상 · 선택

## 확인할 것

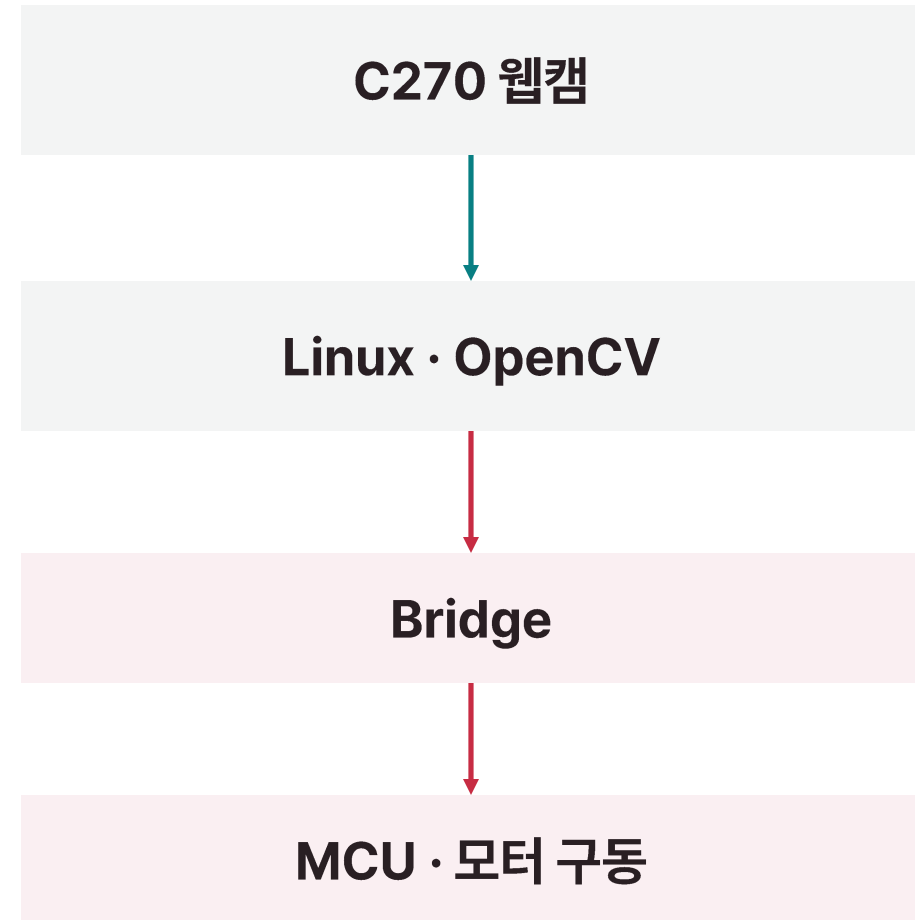
- ☐ 팀 리포지토리에 push
- ☐ sketch.ino의 TODO 완성
- ☐ 실제 주행을 조교에게 확인

# 다음 시간에는 카메라와 적외선 센서를 연결합니다

지금은 정해진 시간만큼  
움직입니다

앞에 무엇이 있는지 읽는 단계가  
아직 들어 있지 않습니다.

다음 단계는 관찰을  
모터 명령으로 연결하는 일입니다.



# 더 궁금하다면

## 속도와 PWM 해상도

analogWriteResolution()을 바꾸면  
최댓값과 255 기반 코드도 함께 바뀝니다.

## 직진이 휘는 이유

두 모터의 차이를 관찰하고  
한쪽 명령에 보정계수를 적용해 봅니다.

## 키보드로 조종하기

Python에서 받은 입력을 Bridge로 전달해  
MCU의 주행 함수를 호출할 수 있습니다.

## UNO Q의 Linux

Terminal에서 ls /, cat /proc/cpuinfo로  
파일과 프로세서 정보를 살펴봅니다.

# 질문과 토론

Q-ROVER 1차시

팀별 실습과 주행 확인

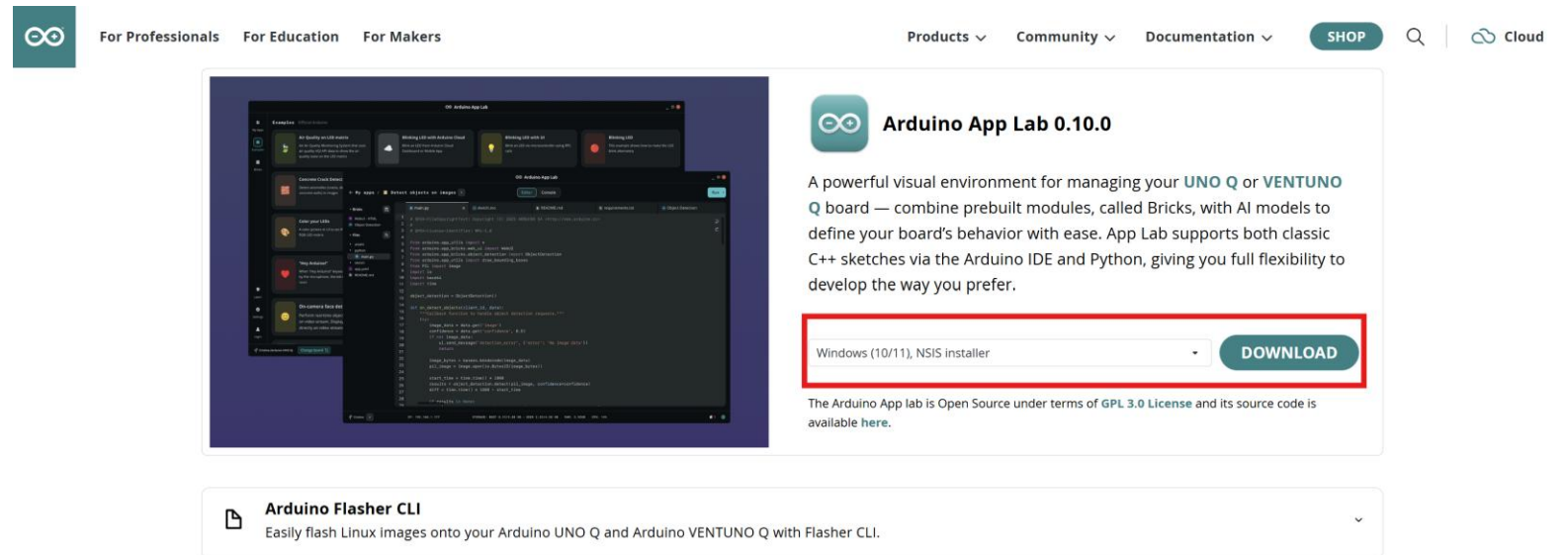


# 설치 참고: App Lab 다운로드

## 운영체제에 맞는 설치 파일 선택

Arduino 공식 소프트웨어 페이지에서  
App Lab을 다운로드합니다.

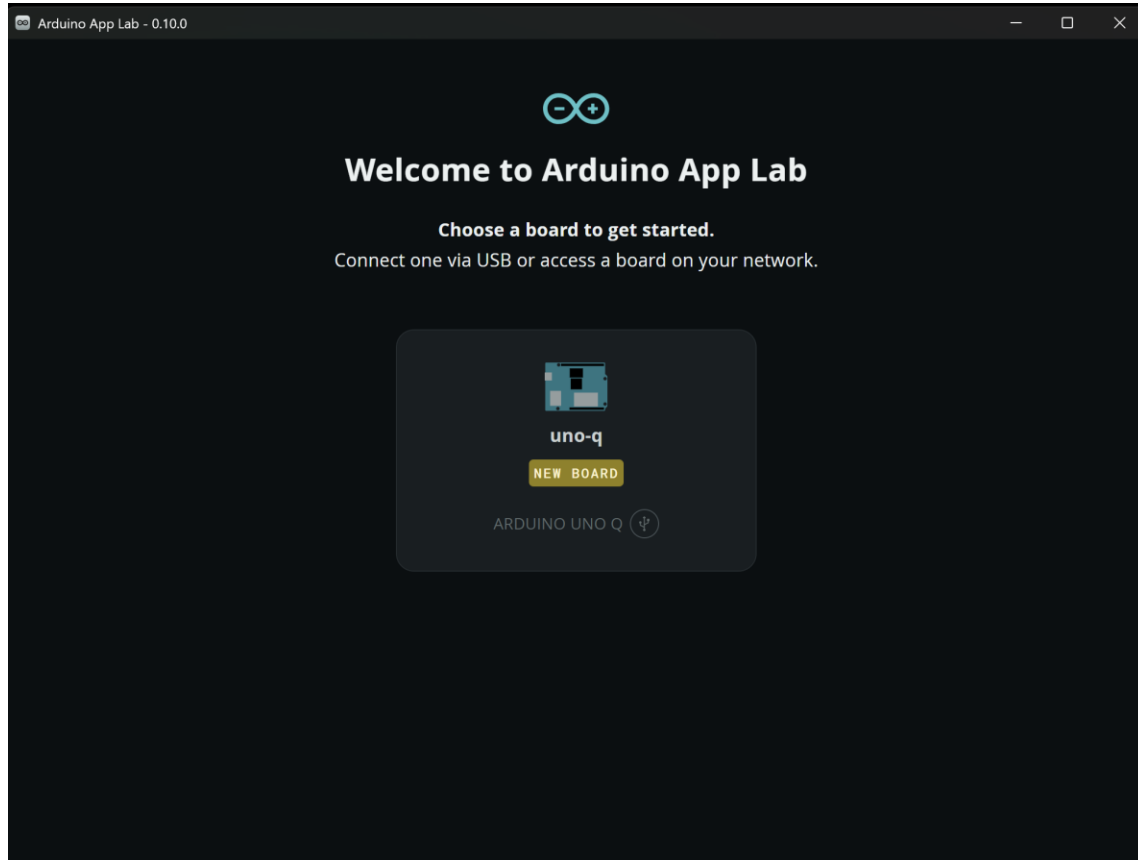
팀당 노트북 한 대로 진행합니다.



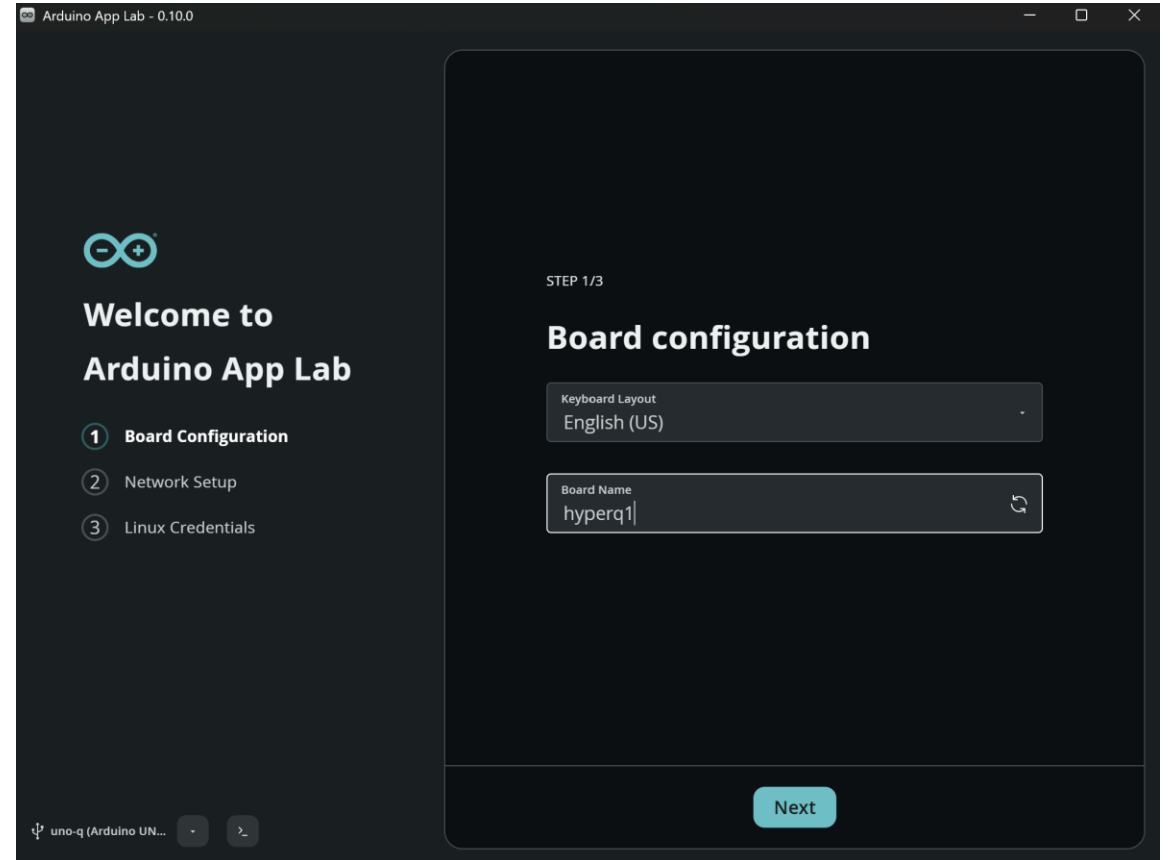
**arduino.cc/en/software**

설치가 끝난 팀은 바로 실습을 진행합니다. 화면은 제공된 설치 당시의 모습입니다.

# 설치 참고: 보드 연결과 이름



1 연결된 UNO Q 선택



2 팀의 보드 이름 설정

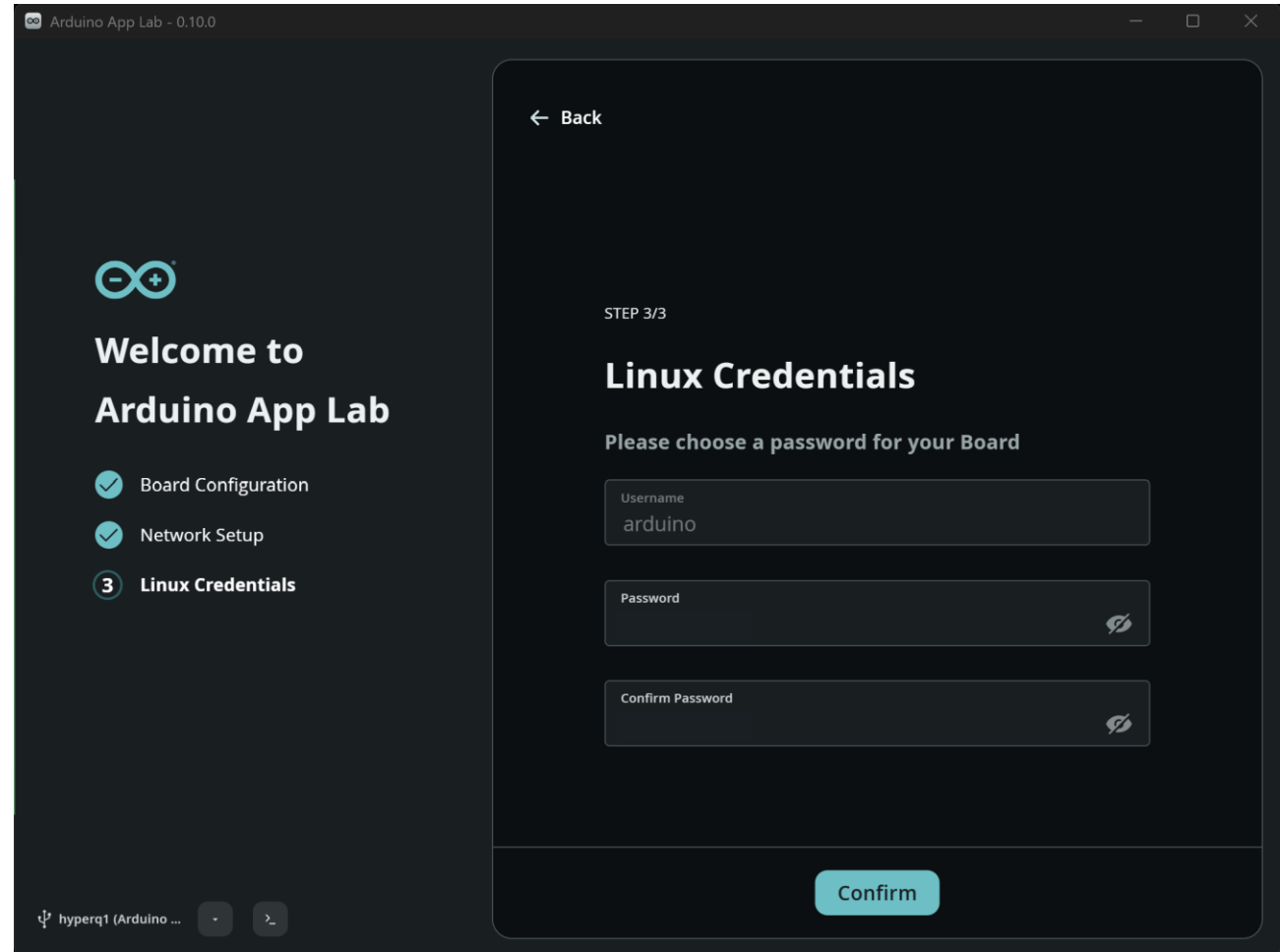
# 설치 참고: Linux 계정 설정

## Linux 계정 설정

보드에 로그인할 계정의  
비밀번호를 정합니다.

팀에서 관리할 값을 입력하고  
확인 후 설정을 완료합니다.

설정이 끝나면  
Q-ROVER-01을 실행합니다.



The screenshot shows the 'Arduino App Lab - 0.10.0' window. On the left, a sidebar lists the setup steps: 'Board Configuration', 'Network Setup', and 'Linux Credentials' (which is selected and numbered 3). The main area is titled 'STEP 3/3 Linux Credentials' and prompts the user to 'Please choose a password for your Board'. It contains three input fields: 'Username' (pre-filled with 'arduino'), 'Password', and 'Confirm Password'. Each password field has a toggle icon for visibility. A 'Confirm' button is at the bottom right. At the bottom left of the window, a status bar shows 'hyperq1 (Arduino ...)' with expand and collapse icons.